

Password Audit Suite Project Manual

Controlled password policy assessment and credential security teaching lab

Project documentation, setup guide, module explanation, demonstrations, tests, and viva preparation

Student: _____ Enrollment number: _____

College: _____ Course and semester: _____

Guide: _____ Academic year: _____

Overview

This manual describes the supplied Python project and gives exact Windows PowerShell commands for running it. The toolkit generates bounded sample word variants, recognizes synthetic hash formats, simulates local candidate comparisons, estimates illustrative search times, analyzes password patterns, and exports redacted reports. It does not extract real Linux or Windows credentials or test live accounts.

Contents

- 1. Certificate and acknowledgement
- 2. Problem, motivation, and objectives
- 3. Scope and implementation map
- 4. Requirements and Windows setup
- 5. Architecture and data flow
- 6. Modules and algorithms
- 7. Complete command reference
- 8. Desktop interface
- 9. Sample execution and output
- 10. Testing and troubleshooting
- 11. Findings and defensive recommendations
- 12. Limitations, future work, and viva
- 13. References and submission checklist

1 Certificate and acknowledgement

Certificate template: This is to certify that _____ completed the project “Password Audit Suite” under the guidance of _____ during the academic year _____. The work was reviewed according to the department requirements. Student, guide, and department signatures: _____.

Acknowledgement template: I thank my project guide and department for guidance, facilities, and feedback during this controlled lab project. Replace these template fields with your verified details before submission.

2 Problem, motivation, and objectives

Weak, reused, or predictable passwords contribute to account takeover and data exposure. An online guessing attack also depends on rate limits and multifactor authentication, while an exposed password database raises a different offline storage question. The project helps a student distinguish these cases using small, reproducible lab examples.

- Generate case, leet, and suffix variants of a sample seed.
- Identify sample hash prefixes and locked account markers without exposing credential material.
- Demonstrate a bounded local dictionary comparison and estimate a mathematical search space.
- Flag short, common, and predictable sample passwords.
- Write redacted JSON and HTML audit reports with mitigation advice.

3 Scope and implementation map

Requested feature	Delivered in this version	Important boundary
Dictionary generator	Case, leet, 1, 123, 2026, ! mutations	At most 5,000 results; no name or DOB harvesting
Linux shadow extraction	Nine-field synthetic fixture parser	Does not open /etc/shadow or return hash strings
Windows SAM extraction	Conceptual discussion only	No SAM or SYSTEM export
Brute force	Bounded local dictionary comparison; search-space formula	No incremental character enumeration or live login attempts
Strength analyzer	Length, classes, common patterns, naive entropy	Heuristic and illustrative, not a security certification
Report	Redacted JSON and optional standalone HTML	Sample CSV itself contains disposable plaintext

4 Requirements and Windows setup

Requirements

- Windows 10 or 11 with Python 3.10 or newer.
- The command-line toolkit uses only Python standard library modules.
- The optional desktop app uses Tkinter, typically included with Windows Python.
- No server, network connection, pip installation, John the Ripper, Hashcat, or administrator rights are required.

Extract the project

Download the ZIP, right-click it in Downloads, select Extract All, then open the extracted Password_Audit_Suite folder. In its File Explorer address bar type powershell and press Enter. The current directory must contain main.py, gui.py, samples, and suite.

```
PS C:\Users\lenovo\Downloads>Password_Audit_Suite_Full_Project> dir
```

```
PS ...> python --version
```

```
PS ...> python main.py -h
```

If your extracted folder has another nested folder, use `cd Password_Audit_Suite` until `dir` shows `main.py`. On Windows, try `py` in place of `python` if that is how Python is installed. The command `python3` is optional and may not exist on some Windows installations.

Why the earlier command printed an error

Running `python3 main.py` with no subcommand starts the parser but gives “the following arguments are required: command.” This means Python found and executed the file. Add one of `dictionary`, `analyze`, `inspect`, `fixture`, `simulate`, `estimate`, or `report`.

```
python main.py report samples/lab_passwords.csv
```

First run and files

The `report` command writes `outputs/audit_report.json`. The `HTML` option writes `outputs/audit_report.html`, which you can open in a browser. Use only the included lab CSV or other disposable examples; never place actual user passwords in a CSV.

5 Architecture and data flow

The program has a CLI dispatcher in `main.py`, reusable functions in `suite/core.py`, an optional desktop window in `gui.py`, synthetic inputs in `samples`, and report outputs in `outputs`. Each command runs locally and finishes without contacting an external service.

Stage	Input	Operation	Output
1	Disposable lab values or CSV	Validate supplied parameters	In-memory values
2	Seed word	Generate bounded mutations	Candidate list
3	Synthetic hash or fixture	Recognize prefix and lock markers	Redacted format metadata
4	Target and candidates	Compare local SHA-256 demo digests	Attempt count and match flag
5	Sample password	Apply heuristic checks	Rating, reasons, naive bits
6	CSV assessment	Assemble findings	Redacted JSON or HTML

Flow: lab input → bounded generation and format inspection → local simulation and analysis → report. The `fixture` command is independent of the CSV report; it does not export hashes to that report.

6 Modules and algorithms

Dictionary generator

variants(seed, limit) normalizes a seed, creates lowercase, title case, uppercase, and selected leet forms, and adds short suffixes. It removes duplicates while keeping first occurrence order. The limit is 1 to 5,000. It does not build a large combinatorial attack list.

Synthetic hash inspection

inspect_hash recognizes \$1\$, \$5\$, \$6\$, and \$y\$ style prefixes as sample md5-crypt, sha256-crypt, sha512-crypt, and yescrypt formats. It also recognizes !, !!, and * lock markers. A 32-character hex value is ambiguous: it may resemble an MD5 or NT hash, so the program does not claim to distinguish them by length alone. inspect_shadow_fixture validates nine colon-separated fields from an explicitly supplied sample file and returns account labels and format metadata only.

Attempt simulation and time estimate

simulate hashes a disposable local target and each candidate with SHA-256 to show a comparison loop. It stops on a match or at a maximum of 100,000 attempts. SHA-256 here is a classroom comparison mechanism, not a recommendation for production password storage. estimate calculates $\text{alphabet_size}^{\text{length}}$ and divides half of that space by an assumed guesses-per-second rate. This assumes uniform random selection and a constant rate, which rarely matches human passwords or real online systems.

6 Modules and algorithms continued

Strength analyzer

analyze checks sample length, observed character classes, a small built-in list of common strings, and predictable sequences or repetition. It calculates a naive bit count as $\text{length} \times \log_2(\text{alphabet size})$. A “stronger” rating means only that none of these particular flags fired. Human choice and reuse are not measured.

Report generation

audit_csv reads label,password rows from a local disposable CSV. The generated report contains each label, length, rating, and reasons, but excludes plaintext. The optional HTML renderer escapes labels and recommendation strings. The original CSV remains plaintext and should be treated as a teaching fixture.

7 Complete command reference

Command	Example	Result
dictionary	python main.py dictionary demo	In-memory mutations printed as JSON
analyze	python main.py analyze	Hidden prompt and heuristic finding

inspect	python main.py inspect '\$6\$salt\$example'	Sample format metadata
fixture	python main.py fixture samples/synthetic_shadow.txt	Redacted synthetic accounts
simulate	python main.py simulate demo123 demo	Local bounded match and attempt count
estimate	python main.py estimate 8 62 1000	Illustrative search space and seconds
report	python main.py report samples/lab_passwords.csv	Redacted JSON report

In PowerShell, use single quotes around literal strings containing dollar signs so PowerShell does not expand them. The analyze command prompts with no echo when you omit its password argument. The simulate command takes a disposable target on the command line, which shell history can retain, so never use real credentials.

Useful options

```
python main.py dictionary demo --limit 12
```

```
python main.py simulate demo123 demo --max-attempts 20
```

```
python main.py report samples/lab_passwords.csv --html outputs/audit_report.html
```

```
python main.py report samples/lab_passwords.csv --out outputs/my_audit.json
```

```
python main.py dictionary -h
```

8 Desktop interface

Start with `python gui.py`. The Tkinter window has tabs for Analyze, Dictionary, Simulate, Inspect, and Audit report. In Analyze, type a disposable example and click Analyze sample. The password field is cleared after the result. The Simulate tab also clears its target field and hides the matched candidate in the display. The Inspect tab can classify a synthetic string or open the included `synthetic_shadow.txt` fixture. The Audit report tab lets you select the sample CSV and save an HTML file.

```
python gui.py
```

If the window does not open, check that your Python installation includes Tkinter by running `python -m tkinter`. A small test window should appear. On Linux, `python3-tk` may need to be installed separately. A headless or remote command prompt may not support desktop windows; the CLI remains available.

9 Sample execution and output

The included samples contain four disposable values. The two short, common examples are flagged weak and the two longer examples receive a stronger heuristic rating. The report does not display

their plaintext values. Screenshots below show generated sample command output, not an audit of real accounts.

```
Audit report – captured sample output

$ python main.py audit-report
Report saved: outputs/audit_report.json
{
  "scope": "Synthetic, disposable lab samples only",
  "sample_count": 4,
  "weak_count": 2,
  "results": [
    {
      "label": "sample_a",
      "length": 11,
      "rating": "weak",
      "reasons": [
        "fewer than 12 characters",
        "limited character variety",
        "contains a common password"
      ]
    },
    {
      "label": "sample_b",
      "length": 17,
      "rating": "stronger",
      "reasons": []
    },
    {
      "label": "sample_c",
      "length": 6,
      "rating": "weak",
      "reasons": [
        "fewer than 12 characters",
        "limited character variety",
        "contains a common password"
      ]
    },
    {
      "label": "sample_d",
      "length": 24,
      "rating": "stronger",
      "reasons": []
    }
  ],
  "recommendations": [
    "Use long unique passwords and a password manager",
    "Require MFA for important accounts",
    "Rate limit online sign-in attempts",
    "Store passwords with salted adaptive hashes such as Argon2id"
  ]
}
```

Figure 1 Redacted sample audit output

```

Attempt simulation – captured sample output

$ python main.py attempt-simulation
{
  "attempts": 3,
  "matched": true,
  "candidate": "demo123",
  "mode": "local SHA-256 teaching demo"
}

```

Figure 2 Local bounded comparison output

9 Sample execution continued

```

Hash format inspection – captured sample output

$ python main.py hash-format-inspection
{
  "format": "sha512-crypt",
  "salted": true,
  "input_redacted": true
}

```

Figure 3 Synthetic hash format identification

```

Time estimate – captured sample output

$ python main.py time-estimate
{
  "search_space": 218340105584896,
  "average_seconds": 109170052792.448,
  "assumption": "Uniform random password; fixed illustrative rate, no rate limiting."
}

```

Figure 4 Illustrative search space estimate

10 Testing and troubleshooting

```
python -m unittest discover -s tests -v
```

The supplied suite has eight checks covering bounded variants, weak and stronger ratings, hash ambiguity, fixture redaction, local match and stop count, estimate arithmetic, report plaintext omission, and HTML label escaping. Run the command from the folder containing main.py.

Problem	Cause	Fix
Missing command error	main.py requires a subcommand	Run <code>python main.py -h</code> or a listed example
Cannot open main.py	Wrong current directory	Use <code>dir</code> , then <code>cd</code> into the folder containing main.py
No module named suite	Running a test from outside project directory	Change into the project root and rerun
CSV file not found	Incorrect relative path	Use <code>samples/lab_passwords.csv</code>

		from project root
python not recognized	Python PATH or install issue	Try <code>py --version</code> ; install Python with PATH enabled
No module named tkinter	Python build lacks desktop bindings	Use CLI or install Tkinter for that Python
Hash \$ signs disappear	PowerShell expanded double-quoted text	Use single quotes around synthetic hash argument

11 Findings and defensive recommendations

The sample audit shows that common patterns can receive a weak rating, while longer samples avoid the limited heuristic flags. Those outcomes are examples, not evidence of any real account security. For a production verifier, current NIST SP 800-63B requires a blocklist of commonly used or compromised passwords and sets minimum lengths based on the authentication context. The toolkit does not implement these full verifier rules. Its character-class flag is a classroom signal rather than a policy mandate.

- Use long, unique passwords and a password manager.
- Reject known common or compromised choices when establishing passwords.
- Use multifactor authentication and rate limits for online sign-in.
- Store passwords using salted adaptive password hashing, such as Argon2id under an appropriate configuration.
- Avoid plaintext credentials in reports, tickets, screenshots, and source control.

12 Limitations and future work

No privileged Linux shadow read, Windows SAM/SYSTEM export, live login requests, large-scale candidate enumeration, or measured cracking performance is implemented. The fixture resembles a shadow file structurally but contains deliberately illustrative strings. The report CSV is for disposable lab data only. A future version could add a policy profile and privacy-preserving breach screening after a separate design and authorization review.

Viva questions

Why does `main.py` need a command?

It uses `argparse` subcommands, each selecting a different function.

Why is a 32-hex value ambiguous?

Different digest formats can share the same textual length.

What is a salt?

A unique value combined with each password before adaptive hashing.

Why is SHA-256 used in the demo but not recommended for storage?

Its fast comparison is convenient for illustration but fast hashes enable rapid offline guesses.

Why is estimated entropy uncertain?

The formula assumes random independent choices, unlike many human passwords.

What does the report omit?

Plaintext sample passwords and fixture hash strings.

How do online and offline guessing differ?

Online systems can rate limit requests; exposed hash verification is performed away from the service.

13 References and submission checklist

NIST, Digital Identity Guidelines SP 800-63B, <https://pages.nist.gov/800-63-4/sp800-63b.html>

OWASP, Password Storage Cheat Sheet,

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP, Authentication Cheat Sheet,

https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Python documentation, argparse, tkinter, hashlib, unittest, <https://docs.python.org/3/library/>

- Fill student, college, guide, and certificate fields.
- Run the exact commands on the submission computer and capture your own screenshots.
- Submit the ZIP project, this manual, generated report, and presentation.
- Explain that real credential extraction and live attacks are outside the delivered simulation.

End of manual.